

Функции и динамическое программирование

Функции

По поведению:

- Функция является **детерминированной**, если при одном и том же заданном входном значении она всегда возвращает один и тот же результат.
- Функция является **недетерминированной**, если она может возвращать различные значения при одном и том же заданном входном значении.

Категории волатильности функций

Категория волатильности - это “обещание” оптимизатору относительно поведения функции:

- `VOLATILE` - может выполнять любые запросы, включая изменение базы данных.
- `STABLE` - не может изменять базу данных и гарантированно возвращает одни и те же результаты при одинаковых аргументах для всех строк в одном операторе.
- `IMMUTABLE` - не может изменять базу данных и гарантированно возвращает одни и те же результаты при одинаковых аргументах навсегда.

Типы возвращаемых значений

По типу возвращаемого значения:

- Скалярная функция;
- Подставляемая табличная функция;
- Многооператорная табличная функция.

Скалярная функция

Синтаксис:

CREATE [OR REPLACE] FUNCTION [имя-схемы.]
имя-функции ([список-объявлений-параметров])
RETURNS скалярный-тип-данных
AS \$\$
 тело-функции
\$\$ [;]

```
CREATE FUNCTION one()  
RETURNS integer AS  
begin  
    SELECT 1 AS result;  
end  
LANGUAGE SQL;
```

```
CREATE          FUNCTION      add_em(integer,  
integer)  
RETURNS integer AS  
$$  
    SELECT $1 + $2;  
$$  
LANGUAGE SQL;
```

```
CREATE FUNCTION getSalary(int)  
RETURNS int AS  
$$  
    SELECT    max(Salary)    AS    result  
FROM foo  
    WHERE fooid = $1;  
$$  
LANGUAGE SQL;
```

Подставляемая табличная функция

Синтаксис:

```
CREATE FUNCTION [ имя-схемы. ] имя-функции ( [
  список-объявлений-параметров ] )
RETURNS SETOF <скалярный тип>
AS $$
[ выражение-выборки ]
$$ [ ; ]
```

```
CREATE FUNCTION list_children(text)
RETURNS SETOF text AS
$$
    SELECT name FROM nodes
    WHERE parent = $1
$$
LANGUAGE SQL;
```

```
CREATE FUNCTION get_foo(id int)
RETURNS foo AS
$$
    RETURN QUERY
    SELECT * FROM foo
    WHERE fooid = id ;
$$ LANGUAGE SQL;
```


Многооператорная функция

Синтаксис:

CREATE FUNCTION [имя-схемы.] имя-функции (
 [список-объявлений-параметров])
 RETURNS TABLE (определение-таблицы)
 AS \$\$
 [SQL- запрос]
 \$\$ [;]

```
create table tab (id int, name text);

insert into tab values(1, 'aaa');
insert into tab values(2, 'bbb');
insert into tab values(3, 'ccc');
insert into tab values(4, 'ddd');
insert into tab values(5, 'eee');
insert into tab values(6, 'fff');
```

```
create or replace function
sum_n_product_with_tab(x int, y int)
returns table(sum int, product int)
AS
$$
begin
    create temp table if not exists
test_calc as
    select id, name
    from tab
    where id > $2;

    return query
    select tab.id, $1 * tab.id
    from test_calc tab;

end
$$ language plpgsql;
```

Управляющие структуры

Использование RETURN
(В функциях,
возвращающих set of
rows):

- NEXT выражение;
- QUERY query;
- QUERY EXECUTE
строка-команды;

```
CREATE OR REPLACE FUNCTION get_all_foo()  
RETURNS SETOF foo AS  
$BODY$  
DECLARE  
    r foo%rowtype;  
BEGIN  
    FOR r IN SELECT * FROM foo  
              WHERE fooid > 0  
    LOOP  
        -- здесь возможна обработка данных  
        -- возвращается текущая строка запроса  
        RETURN NEXT r;  
    END LOOP;  
    RETURN;  
END;  
$BODY$  
LANGUAGE plpgsql;  
  
SELECT * FROM get_all_foo();
```


Метаданные (information_schema)

Информационная схема (information_schema) состоит из набора представлений, содержащих информацию об объектах, определённых в текущей базе данных.

Информационная схема описана в стандарте SQL и поэтому можно рассчитывать на её переносимость и стабильность — в отличие от системных каталогов, которые привязаны к PostgreSQL, и моделируются, отталкиваясь от реализации.

- information_schema.schemata
- information_schema.tables
- information_schema.columns
- information_schema.routines
- information_schema.triggers
- information_schema.views
- information_schema.
table_constraints
- information_schema.
check_constraints